

Project Topics to Be Included

Each student project **must include the following topics and functionalities:**

1. Project Setup

- Initialize a Node.js project
- Install and configure required dependencies
- Organize the project using a clean folder structure (routes, controllers, models, middleware)

2. Server Creation

- Create an Express server
- Configure middleware (JSON parsing, error handling)
- Use environment variables for configuration

3. Database Connection

- Connect the application to MongoDB
- Use **Mongoose** for database interaction
- Handle database connection errors

4. Application Routing

- Create RESTful API routes
- Use proper HTTP methods (GET, POST, PUT/PATCH, DELETE)
- Structure routes based on resources (users, main entities)

5. User Authentication

- Implement user registration
- Implement user login
- Hash passwords using **bcrypt**
- Generate authentication tokens using **JSON Web Token (JWT)**

6. User Authorization

- Protect private routes using JWT middleware
- Allow users to access only their own data
- Implement role-based access if needed (optional)

7. Data Modeling

- Create database schemas using Mongoose
- Define relationships between collections
- Use required fields and default values

8. Data Validation

- Validate request data on the server side
- Prevent invalid or empty data from being saved
- Return meaningful validation error messages

9. CRUD Operations

- Create new records (POST)
- Read data (GET all / GET by ID)
- Update existing records (PUT / PATCH)
- Delete records (DELETE)

10. Profile or Main Entity Management

- Create a main profile or core entity related to the project
- Allow users to create and update their own profile or main data
- Link the profile/entity to the authenticated user

11. Data Ownership & Security

- Ensure users can access only their own resources
- Prevent unauthorized access or modification
- Handle forbidden actions correctly

14. Error Handling

- Handle application and database errors
- Use consistent error response structure
- Avoid application crashes due to unhandled errors

15. API Testing

- Test endpoints using Postman or similar tools
- Verify authentication and authorization flow
- Validate CRUD functionality